



SY5 – Systèmes d'exploitation

Examen de 1^{re} session – 8 janvier 2025

Durée : 3 heures

Aucun document autorisé
Appareils électroniques éteints et rangés

Tous les programmes demandés doivent être écrits en C, de la manière la plus lisible possible, c'est-à-dire bien indentés et commentés aux endroits où cela paraît nécessaire.

En revanche, s'agissant d'une épreuve sur papier, on n'attend pas un code irréprochable; en particulier il est inutile d'indiquer les inclusions nécessaires, et vous pouvez vous contenter d'une gestion minimale des erreurs dans les cas non problématiques (par exemple avec un simple commentaire).

L'utilisation de la bibliothèque `stdio` doit se limiter à la gestion des erreurs, au formatage de chaînes de caractères, et à un éventuel affichage en fin de programme.

Exercice 1 : histoires de famille

On considère une commande « zombies » dont l'exécution provoque la situation suivante :

```
poulalho@lulu:SY5$ ps j ; ./zombies & sleep 5 ; ps j ; sleep 5 ; ps j
PPID   PID   PGID   SID TTY      TPGID STAT  UID   TIME COMMAND
3960545 3960561 3960561 3960561 pts/23   3966434 Ss      8159   0:00 -bash
3960561 3966434 3966434 3960561 pts/23   3966434 R+      8159   0:00 ps j
PPID   PID   PGID   SID TTY      TPGID STAT  UID   TIME COMMAND
3960545 3960561 3960561 3960561 pts/23   3966443 Ss      8159   0:00 -bash
3960561 3966435 3966435 3960561 pts/23   3966443 S      8159   0:00 ./zombies
3966435 3966438 3966435 3960561 pts/23   3966443 S      8159   0:00 ./zombies
1 3966439 3966435 3960561 pts/23   3966443 S      8159   0:00 sleep 7
3966438 3966440 3966435 3960561 pts/23   3966443 Z      8159   0:00 [sleep] <defunct>
1 3966441 3966435 3960561 pts/23   3966443 S      8159   0:00 sleep 7
3966438 3966442 3966435 3960561 pts/23   3966443 Z      8159   0:00 [sleep] <defunct>
3960561 3966443 3966443 3960561 pts/23   3966443 R+      8159   0:00 ps j
PPID   PID   PGID   SID TTY      TPGID STAT  UID   TIME COMMAND
3960545 3960561 3960561 3960561 pts/23   3966470 Ss      8159   0:00 -bash
3960561 3966435 3966435 3960561 pts/23   3966470 S      8159   0:00 ./zombies
3960561 3966470 3966470 3960561 pts/23   3966470 R+      8159   0:00 ps j
```

1. Dessiner la généalogie des processus créés par l'exécution de « zombies ».
2. Expliquer précisément l'état de chacun de ces processus au bout de 5 secondes (i.e. lors de la deuxième exécution de « ps j »).
3. Que s'est-il passé durant les 5 secondes suivantes (i.e. entre la 2^e et la 3^e exécutions de « ps j ») ?
4. Écrire un programme `zombies.c` ayant exactement le comportement observé.

Exercice 2 : décompte d'erreurs

Écrire un programme qui, étant donné deux références `ref1` et `ref2` passées en paramètre, vérifie qu'il s'agit des références de deux fichiers accessibles, puis calcule leur *distance de Hamming*, définie comme le nombre de positions i où les deux fichiers diffèrent : soit `ref1` et `ref2` contiennent un octet différent en position i , soit l'un des deux fichiers est de taille strictement inférieure à i et l'autre supérieure ou égale à i . En particulier, deux fichiers sont à distance 0 si et seulement s'ils sont identiques; si `ref2` résulte de la copie de `ref1`, leur distance de Hamming est une mesure des éventuelles erreurs de recopie.

Ce programme a vocation à comparer de (très) gros fichiers, et doit donc être efficace quelle que soit la taille des fichiers considérés.

Exercice 3 : nettoyage d'arborescence

Dans cet exercice, il est interdit d'exécuter une autre commande que le programme à écrire (en particulier, pas de « `find` »), et d'utiliser les bibliothèques `ftw` et `fts`.

Écrire un programme qui supprime tous les fichiers vides et sous-répertoires vides contenus dans l'arborescence dont le répertoire courant est la racine, et plus généralement toutes ses sous-arborescences constituées uniquement de répertoires ne contenant aucun fichier non vide.

Pour cela, il pourra être utile d'écrire une fonction récursive `int nettoie(char *ref)` réalisant ce nettoyage dans l'arborescence de racine `ref`, et renvoyant 0 ou 1 selon que le nettoyage aboutit à la suppression de `ref` ou non.

(On supposera pour simplifier que l'arborescence ne contient pas de liens symboliques)

Exercice 4 : liste des ouvertures de fichiers

La commande « `ls -l` » permet de lister les entrées de la table des fichiers ouverts; avec en paramètre le nom d'un fichier, elle se limite aux ouvertures de celui-ci. Par exemple, si je crée un tube nommé `/tmp/montube` puis que je lance deux processus exécutant respectivement « `cat /tmp/montube` » et « `cat > /tmp/montube` » depuis deux terminaux différents, j'obtiens :

```
poulalho@lulu:SY5$ ls -l /tmp/montube
COMMAND  PID   USER   FD  TYPE DEVICE SIZE/OFF  NODE NAME
cat       3656042 poulalho 3r  FIFO  0,65      0t0 56215 /tmp/montube
cat       3656063 poulalho 1w  FIFO  0,65      0t0 56215 /tmp/montube
```

Un même processus peut apparaître plusieurs fois; par exemple si un job exécutant « `vim` » tourne dans le terminal `/dev/pts/19` d'où a été lancé le processus 3656063, cela donne :

```
poulalho@lulu:SY5$ ls -l /dev/pts/19
COMMAND  PID   USER   FD  TYPE DEVICE SIZE/OFF  NODE NAME
bash     3621352 poulalho 0u  CHR 136,19      0t0 22 /dev/pts/19
bash     3621352 poulalho 1u  CHR 136,19      0t0 22 /dev/pts/19
bash     3621352 poulalho 2u  CHR 136,19      0t0 22 /dev/pts/19
bash     3621352 poulalho 255u CHR 136,19      0t0 22 /dev/pts/19
vim       3655729 poulalho 0u  CHR 136,28      0t0 22 /dev/pts/19
vim       3655729 poulalho 1u  CHR 136,28      0t0 22 /dev/pts/19
vim       3655729 poulalho 2u  CHR 136,28      0t0 22 /dev/pts/19
cat       3656063 poulalho 0u  CHR 136,19      0t0 22 /dev/pts/19
cat       3656063 poulalho 2u  CHR 136,19      0t0 22 /dev/pts/19
```

En particulier, la colonne « `FD` » indique les descripteurs et les modes associés à chaque ouverture (« `u` » indiquant une ouverture en lecture et en écriture).

1. Expliquer les valeurs correspondant aux ouvertures de `/tmp/montube` et `/dev/pts/19` par les deux processus « `cat` » (lecteur et écrivain), en précisant notamment lequel des deux a été lancé sur le terminal `/dev/pts/19`.

L'option « `-t` » permet de n'obtenir que la liste (sans doublon) des processus possédant (au moins) une ouverture du fichier :

```
poulalho@lulu:SY5$ ls -l -t /tmp/montube
3656042
3656063
poulalho@lulu:SY5$ ls -l -t /dev/pts/19
3621352
3655729
3656063
```

2. Écrire un programme `cpof.c` utilisant « `ls of` » pour compter les processus possédant une ouverture sur un fichier dont le nom est passé en paramètre. Par exemple, dans la situation précédente, on attend l'affichage suivant :

```
poulalho@lulu:SY5$ ./cpof /tmp/montube
2
poulalho@lulu:SY5$ ./cpof /dev/pts/19
3
```

Votre programme ne doit exécuter aucune autre commande que « `ls of` ».

Exercice 5 : un problème de synchronisation

1. Quels affichages le programme suivant peut-il produire ?

```
int main() {
    for (int i=1; i<4; i++) {
        if (fork() == 0) { printf("%d... ", i); exit(0); }
    }
    printf("partez! ");
}
```

Dans les questions suivantes, on cherche à assurer un affichage précis *sans modifier la structure* du programme ci-dessus (donc uniquement en *ajoutant* des instructions, mais sans changer la généalogie, ni le message affiché par chaque processus).

- Proposer une modification minimale du programme permettant d'assurer que l'affichage soit toujours "1... 2... 3... partez! " (sauf en cas d'erreur de `fork()`, naturellement).
- Cette solution est-elle adaptable pour assurer un autre affichage précis ?
- Proposer une solution pour assurer l'affichage "3... 2... 1... partez! ", en synchronisant les processus à l'aide de `tube(s)` anonyme(s).
- Proposer une autre solution de synchronisation reposant sur l'envoi de signaux. Expliquer en particulier quand le(s) gestionnaire(s) doit(vent) être mis en place. S'il y a un risque de blocage, expliquer comment l'éviter.

Exercice 6 : devinette

Écrire un programme qui simule à l'aide de tubes un jeu de devinette entre un (processus) père et ses N fils : au début du jeu, le père tire une valeur aléatoire M entre 0 et $2^{31} - 1$, puis une deuxième valeur aléatoire S (qu'il garde secrète) entre 0 et M ; il dévoile alors la valeur M à ses fils, et chaque fils tente alors d'être le premier à deviner S .

Pour cela, chaque fils envoie à son père des tentatives successives, et le père lui répond en indiquant si la tentative est trop grande ou trop petite, jusqu'à ce qu'un fils devine S . Le père interrompt alors le jeu et annonce le vainqueur.

La transmission des tentatives des fils et des réponses du père se fera par `tube(s)` anonyme(s). Le père ne doit favoriser aucun fils, et toujours répondre aux propositions dans l'ordre où elles sont émises. Aucune synchronisation particulière n'est requise entre les fils – chacun s'efforce d'être aussi rapide que possible, tout en respectant une seule et unique règle : attendre la réponse du père avant de faire une nouvelle tentative, qui doit être cohérente avec les indices qu'il a précédemment reçus. Pour s'assurer qu'aucun fils ne tente de contourner cette règle, le père vérifiera que chaque nouvelle tentative d'un fils est cohérente avec la série de ses tentatives précédentes.

Préciser en particulier le nombre de tubes nécessaires, le format des messages, et le mécanisme permettant aux fils de détecter la fin du jeu.

Petit mémento de fonctions et structures utiles

```
int open(const char *pathname, int flags /*, mode_t mode */);
int openat(int dirfd, const char *pathname, int flags /*, mode_t mode */);
```

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
off_t lseek(int fd, off_t offset, int whence);
```

```
DIR *opendir(const char *name);
DIR *fdopendir(int fd);
struct dirent *readdir(DIR *dirp);
int closedir(DIR *dirp);
void rewinddir(DIR *dirp);
struct dirent { /* contient entre autres : */
    ino_t d_ino; /* Inode number */
    char d_name[]; /* Null-terminated filename */
};
```

```
int lstat(const char *pathname, struct stat *statbuf);
int stat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
int fstatat(int dirfd, const char *restrict pathname, struct stat *restrict statbuf, int flags);
```

```
struct stat { /* contient entre autres : */
    dev_t st_dev; /* ID of device containing file */
    ino_t st_ino; /* Inode number */
    mode_t st_mode; /* File type and mode */
    off_t st_size; /* file size, in bytes */
};
```

```
int unlink(const char *pathname);
int unlinkat(int dirfd, const char *pathname, int flags);
int rmdir(const char *pathname);
```

```
int execlp(const char *file, const char *arg, ... /* (char *)NULL */);
int execvp(const char *file, char *const argv[]);
```

```
int kill(pid_t pid, int sig);
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
struct sigaction { /* contient entre autres : */
    void (*sa_handler)(int);
    sigset_t sa_mask;
    int sa_flags;
};
```

```
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int sigsuspend(const sigset_t *mask);
int sigwait(const sigset_t *restrict set, int *restrict sig);
```

```
long random(void);
void srand(unsigned int seed);
```